Loan Approval Classification

นางสาว เขมิกา จันทร์งาม รหัสนิสิต 6530200037

นางสาว ปาณิสรา วิจารณ์ รหัสนิสิต 6530200274

นางสาว เพ็ญพิชชา ไพรวัลย์ รหัสนิสิต 6530200312

นางสาว มนัสวี ปิยะโสภาสกุล รหัสนิสิต 6530200371

นางสาว ภัทราลดา พบที่พึ่ง รหัสนิสิต 6530200754

หมู่เรียนบรรยาย 800 ปีการศึกษา 2567

รายวิชา Introduction to Machine Learning รหัสวิชา 01418362

คณะวิทยาศาสตร์ ศรีราชา สาขาวิทยาการคอมพิวเตอร์

มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตศรีราชา

Data Set

1. Data Source

ชุดข้อมูลนี้เป็นเวอร์ชันสังเคราะห์ที่ได้รับแรงบันดาลใจจากชุดข้อมูล Credit Risk ดั้งเดิมบน Kaggle และได้เพิ่มตัวแปรเพิ่มเติมโดยอิงจากข้อมูลความเสี่ยงทางการเงินสำหรับการอนุมัติสินเชื่อ ใช้เทคนิค SMOTENC เพื่อจำลองจุดข้อมูลใหม่เพื่อขยายจำนวนตัวอย่าง ชุดข้อมูลนี้ถูกออกแบบมาให้มีทั้งคุณลักษณะแบบ categorical และแบบ continuous features.

2. Metadata

ชุดข้อมูลนี้ประกอบด้วย 45,000 แถว และ 14 คอลัมน์ ซึ่งอธิบายไว้ดังต่อไปนี้

Column	คำอธิบาย	Data Type
person_age	อายุของบุคคล	Float
person_gender	เพศของบุคคล	Categorical
person_education	ระดับการศึกษาสูงสุด	Categorical
person_income	รายได้ต่อปี	Float
person_emp_exp	ประสบการณ์การทำงาน (ปี)	Integer
person_home_ownership	สถานะการเป็นเจ้าของที่อยู่อาศัย (เช่น เช่า, เป็นเจ้าของ, ผ่อนชำระ)	Categorical
loan_amnt	จำนวนเงินกู้ที่ร้องขอ	Float
loan_intent	วัตถุประสงค์ของการขอเงินกู้	Categorical
loan_int_rate	อัตราดอกเบี้ยเงินกู้	Float
loan_percent_income	จำนวนเงินกู้เป็นเปอร์เซ็นต์ของรายได้ต่อปี	Float
cb_person_cred_hist_length	ระยะเวลาเครดิตในประวัติ (ปี)	Float
credit_score	คะแนนเครดิตของบุคคล	Integer
previous_loan_defaults_on_file	ตัวบ่งชี้การผิดนัดชำระเงินกู้ก่อนหน้า	Categorical
loan_status (target variable)	สถานะการอนุมัติเงินกู้: 1 = อนุมัติ; 0 = ปฏิเสธ	Integer

Source Code

1. ติดตั้งโปรแกรมที่เกี่ยวข้อง

1.1 Download and install java JDK 8

1.2 Download and install Apache Spark 3.5.3

1.3 Install findspark library

```
[26]  1 !apt-get install openjdk-8-jdk-headless -qq > /dev/null # Download and install java JDK 8
```

```
[27]  1 # Download and install Apache Spark
      2 !pip install pyspark py4j
      3 from pyspark.sql import SparkSession
      4 spark = SparkSession.builder.appName('test_pyspark').getOrCreate()
      5 df = spark.read.csv("/content/sample_data/california_housing_test.csv", header=True, inferSchema=True)
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.11/dist-packages (3.5.4)
Requirement already satisfied: py4j in /usr/local/lib/python3.11/dist-packages (0.10.9.7)
```

```
[28]  1 !pip install -q findspark # Install findspark library
```

2. Import Python Libraries และ กำหนด path ของโปรแกรม Java และ Apache Spark ติดตั้งไว้ใน Notebook

```
[29]  1 import os
      2 import sys
      3 import time
      4 import shutil
      5 os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
      6 os.environ["SPARK_HOME"] = "/content/spark-3.5.3-bin-hadoop3"
```

3. Import findspark และ pyspark.sql

```
[30]  1 import findspark
      2 findspark.init()
      3 from pyspark.sql import SparkSession
      4 from pyspark.sql import SQLContext
      5 sc = SparkSession.builder.appName("loan").getOrCreate()
```

4. สร้างตัวแปร data จากไฟล์ csv ที่มีชื่อว่า "loan.csv"

```
1 data_file = '/content/loan.csv'
2 data = sc.read.csv(data_file, header = True, inferSchema = True)
3 print('Total Records = {}'.format(data.count()))
4 data.show()
5 data.printSchema()
```

แสดงจำนวนแถวทั้งหมด ซึ่งมีจำนวน 45000 แถว และแสดงตัวอย่างข้อมูล 20 แถว และแสดงโครงสร้างของข้อมูล

```
Total Records = 45000
+----------+-------------+----------------+-------------+---------------+----------------------+----------+-----------------+-------------+-----------------
|person_age|person_gender|person_education|person_income|person_emp_exp|person_home_ownership|loan_amnt|      loan_intent|loan_int_rate|loan_percent_i
+----------+-------------+----------------+-------------+---------------+----------------------+----------+-----------------+-------------+-----------------
|        22|       female|          Master|        71948|              0|                  RENT|     35000|         PERSONAL|        16.02|
|        21|       female|     High School|        12282|              0|                   OWN|      1000|        EDUCATION|        11.14|
|        25|       female|     High School|        12438|              3|              MORTGAGE|      5500|          MEDICAL|        12.87|
|        23|       female|        Bachelor|        79753|              0|                  RENT|     35000|          MEDICAL|        15.23|
|        24|         male|          Master|        66135|              1|                  RENT|     35000|          MEDICAL|        14.27|
|        21|       female|     High School|        12951|              0|                   OWN|      2500|          VENTURE|         7.14|
|        26|       female|        Bachelor|        93471|              1|                  RENT|     35000|        EDUCATION|        12.42|
|        24|       female|     High School|        95550|              5|                  RENT|     35000|          MEDICAL|        11.11|
|        24|       female|       Associate|       100684|              3|                  RENT|     35000|         PERSONAL|          8.9|
|        21|       female|     High School|        12739|              0|                   OWN|      1600|          VENTURE|        14.74|
|        22|       female|     High School|       102985|              0|                  RENT|     35000|          VENTURE|        10.37|
|        21|       female|       Associate|        13113|              0|                   OWN|      4500|  HOMEIMPROVEMENT|         8.63|
|        23|         male|        Bachelor|       114860|              3|                  RENT|     35000|          VENTURE|         7.9|
|        26|         male|          Master|       130713|              0|                  RENT|     35000|        EDUCATION|        18.39|
|        23|       female|       Associate|       138998|              0|                  RENT|     35000|        EDUCATION|         7.9|
|        23|       female|          Master|       600891|              5|              MORTGAGE|     30000|DEBTCONSOLIDATION|        10.65|
|        23|         male|        Bachelor|       144943|              0|                  RENT|     35000|        EDUCATION|         7.9|
|        23|       female|     High School|       111369|              0|                  RENT|     35000|          MEDICAL|         20.0|
|        23|         male|        Bachelor|       136628|              0|                  RENT|     35000|DEBTCONSOLIDATION|        18.25|
|        24|       female|          Master|        14283|              1|              MORTGAGE|      1750|        EDUCATION|        10.99|
+----------+-------------+----------------+-------------+---------------+----------------------+----------+-----------------+-------------+-----------------
```

```
root
 |-- person_age: integer (nullable = true)
 |-- person_gender: string (nullable = true)
 |-- person_education: string (nullable = true)
 |-- person_income: integer (nullable = true)
 |-- person_emp_exp: integer (nullable = true)
 |-- person_home_ownership: string (nullable = true)
 |-- loan_amnt: integer (nullable = true)
 |-- loan_intent: string (nullable = true)
 |-- loan_int_rate: double (nullable = true)
 |-- loan_percent_income: double (nullable = true)
 |-- cb_person_cred_hist_length: integer (nullable = true)
 |-- credit_score: integer (nullable = true)
 |-- previous_loan_defaults_on_file: string (nullable = true)
 |-- loan_status: integer (nullable = true)
```

```
loan_status = data.groupBy('loan_status').count()
print(loan_status.show())
```

```
+-----------+-----+
|loan_status|count|
+-----------+-----+
|          1|10000|
|          0|35000|
+-----------+-----+

None
```

การ Group รวม Class ผลลัพธ์นั้นก็คือ loan_status โดย 1 คือ Approve และ 0 คือ Reject และ เลือกคอลัมน์ที่สนใจ และ แสดงโครงสร้างของข้อมูล

```
[33] df = data.select('person_age','person_gender','person_education','person_income','person_emp_exp','person_home_ownership','loan_amnt','loan_intent
     cols = df.columns
     df.printSchema()
```

```
root
 |-- person_age: integer (nullable = true)
 |-- person_gender: string (nullable = true)
 |-- person_education: string (nullable = true)
 |-- person_income: integer (nullable = true)
 |-- person_emp_exp: integer (nullable = true)
 |-- person_home_ownership: string (nullable = true)
 |-- loan_amnt: integer (nullable = true)
 |-- loan_intent: string (nullable = true)
 |-- loan_int_rate: double (nullable = true)
 |-- loan_percent_income: double (nullable = true)
 |-- cb_person_cred_hist_length: integer (nullable = true)
 |-- credit_score: integer (nullable = true)
 |-- previous_loan_defaults_on_file: string (nullable = true)
 |-- loan_status: integer (nullable = true)
```

5. สร้าง VectorAssembler

```
[34]
from pyspark.ml.feature import OneHotEncoder, StringIndexer, VectorAssembler
categoricalColumns = ['person_gender','person_education','person_home_ownership','loan_intent','previous_loan_defaults_on_file']

stages = []

for categoricalCol in categoricalColumns:
  stringIndexer = StringIndexer(inputCol = categoricalCol, outputCol = categoricalCol + 'Index')
  encoder = OneHotEncoder(inputCols=[stringIndexer.getOutputCol()], outputCols=[categoricalCol + "classVec"])
  stages += [stringIndexer, encoder]

label_stringIdx = StringIndexer(inputCol = 'loan_status', outputCol = 'label')
stages += [label_stringIdx]

numericCols = ['person_age','person_income','person_emp_exp','loan_amnt','loan_int_rate','loan_percent_income','cb_person_cred_hist_length','credi
assemblerInputs = [c + "classVec" for c in categoricalColumns] + numericCols
assembler = VectorAssembler(inputCols=assemblerInputs, outputCol="features")
stages += [assembler]
```

จัดข้อมูลให้อยู่รูปของ Vector เพื่อส่งไปเทรนโมเดลแบ่งเป็น 2 กลุ่ม

1. Categorical Data แปลงด้วย One Hot Encoder

2. Numberical Data หรือคอลัมน์ที่เป็นตัวเลขเอาไปเชื่อมต่อกับตัว Categorical Data ที่แปลงด้วย One Hot Encoder แล้ว

เสร็จแล้วทำเป็น Feature ออกมา แล้วแสดงรายการข้อมูลใน Feature ที่จะใช้ในการเทรนโมเดล

```
[35]
from pyspark.ml import Pipeline
pipeline = Pipeline(stages = stages)
pipelineModel = pipeline.fit(df)
df = pipelineModel.transform(df)
selectedCols = ['label', 'features'] + cols
df = df.select(selectedCols)
df.show(5)
df.printSchema()
```

```
+-----+--------------------+----------+-------------+----------------+-------------+--------------+---------------------+---------+-----------+------
|label|            features|person_age|person_gender|person_education|person_income|person_emp_exp|person_home_ownership|loan_amnt|loan_intent|loan_in
+-----+--------------------+----------+-------------+----------------+-------------+--------------+---------------------+---------+-----------+------
|  1.0|(22,[4,5,11,14,15...|        22|       female|          Master|        71948|             0|                 RENT|    35000|   PERSONAL|
|  0.0|(22,[3,7,8,13,14,...|        21|       female|     High School|        12282|             0|                  OWN|     1000|  EDUCATION|
|  1.0|(22,[3,6,9,14,15,...|        25|       female|     High School|        12438|             3|             MORTGAGE|     5500|    MEDICAL|
|  1.0|(22,[1,5,9,14,15,...|        23|       female|        Bachelor|        79753|             0|                 RENT|    35000|    MEDICAL|
|  1.0|(22,[0,4,5,9,14,1...|        24|         male|          Master|        66135|             1|                 RENT|    35000|    MEDICAL|
+-----+--------------------+----------+-------------+----------------+-------------+--------------+---------------------+---------+-----------+------
only showing top 5 rows
```

```
root
 |-- label: double (nullable = false)
 |-- features: vector (nullable = true)
 |-- person_age: integer (nullable = true)
 |-- person_gender: string (nullable = true)
 |-- person_education: string (nullable = true)
 |-- person_income: integer (nullable = true)
 |-- person_emp_exp: integer (nullable = true)
 |-- person_home_ownership: string (nullable = true)
 |-- loan_amnt: integer (nullable = true)
 |-- loan_intent: string (nullable = true)
 |-- loan_int_rate: double (nullable = true)
 |-- loan_percent_income: double (nullable = true)
 |-- cb_person_cred_hist_length: integer (nullable = true)
 |-- credit_score: integer (nullable = true)
 |-- previous_loan_defaults_on_file: string (nullable = true)
 |-- loan_status: integer (nullable = true)
```

6. แบ่งตารางข้อมูลเป็น training data และ test data

```
[36]
train, test = df.randomSplit([0.7, 0.3], seed = 2018)
print("Training Dataset Count: " + str(train.count()))
print("Test Dataset Count: " + str(test.count()))
```

```
Training Dataset Count: 31569
Test Dataset Count: 13431
```

แบ่งข้อมูลเทรนและเทสออกเป็น 70:30

7. Train Model

7.1 Train: Logistic Regression

7.2 Train: Naive Bayes

7.3 Train: Random forest

```
[37]
from pyspark.ml.classification import LogisticRegression, NaiveBayes, RandomForestClassifier, LinearSVC
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Train: Logistic Regression
lr = LogisticRegression(featuresCol = 'features', labelCol = 'label', maxIter=10)
lrModel = lr.fit(train)

# Train: Naive Bayes
nb = NaiveBayes(smoothing=1.0, modelType="multinomial")
nbModel = nb.fit(train)

# Train: Random forest
rf = RandomForestClassifier(featuresCol = 'features', labelCol = 'label')
rfModel = rf.fit(train)
```

สร้างโมเดลทั้งหมด 3 โมเดลได้แก่ Logistic Regression, Naive Bayes, และ Random forest

8. Test Model

ทดสอบโมเดลทั้ง 3

8.1 Test: Logistic Regression

```python
# Test: Logistic Regression Model with test data
prediction_logistic_regression_model = lrModel.transform(test)
prediction_logistic_regression_model.select('previous_loan_defaults_on_file', 'loan_percent_income', 'loan_int_rate', 'person_home_ownership', 'la
from pyspark.ml.evaluation import BinaryClassificationEvaluator
evaluator = BinaryClassificationEvaluator()
print('Test Area Under ROC', evaluator.evaluate(prediction_logistic_regression_model))
```

```
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
|previous_loan_defaults_on_file|loan_percent_income|loan_int_rate|person_home_ownership|label|       rawPrediction|prediction|         probability|
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
|                          Yes|               0.14|        15.02|                 RENT|  0.0|[4.77911380531798...|       0.0|[0.99166658660743...|
|                          Yes|               0.09|         6.99|                 RENT|  0.0|[9.14166988873763...|       0.0|[0.99989290314985...|
|                          Yes|               0.16|        10.56|                 RENT|  0.0|[7.20511645261109...|       0.0|[0.99925777561331...|
|                          Yes|               0.05|        10.37|                 RENT|  0.0|[7.60302862308530...|       0.0|[0.99950131076746...|
|                          Yes|               0.08|        13.92|                 RENT|  0.0|[6.80284402744746...|       0.0|[0.99889062004986...|
|                          Yes|               0.12|        11.01|                 RENT|  0.0|[6.12886391102374...|       0.0|[0.99782568280156...|
|                          Yes|               0.04|        10.59|                 RENT|  0.0|[8.41952417107158...|       0.0|[0.99977952906069...|
|                          Yes|               0.19|        14.27|                 RENT|  0.0|[4.32599610641910...|       0.0|[0.98695212318419...|
|                          Yes|               0.09|         9.99|                 RENT|  0.0|[7.43823606558383...|       0.0|[0.99941202387500...|
|                          Yes|               0.15|        14.96|                 RENT|  0.0|[4.42146546826231...|       0.0|[0.98812607490481...|
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
only showing top 10 rows

Test Area Under ROC 0.9541986712240025
```

8.2 Test: Naive Bayes

```python
# Test: Naive Bayes Model with test data
prediction_native_bayes = nbModel.transform(test)
prediction_native_bayes.select('previous_loan_defaults_on_file', 'loan_percent_income', 'loan_int_rate', 'person_home_ownership', 'label', 'rawPre
from pyspark.ml.evaluation import BinaryClassificationEvaluator
evaluator = BinaryClassificationEvaluator()
print('Test Area Under ROC', evaluator.evaluate(prediction_native_bayes))
```

```
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
|previous_loan_defaults_on_file|loan_percent_income|loan_int_rate|person_home_ownership|label|       rawPrediction|prediction|         probability|
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
|                          Yes|               0.14|        15.02|                 RENT|  0.0|[-25218.092797486...|       0.0|[1.0,5.6786599089...|
|                          Yes|               0.09|         6.99|                 RENT|  0.0|[-21778.029236245...|       0.0|            [1.0,0.0]|
|                          Yes|               0.16|        10.56|                 RENT|  0.0|[-34113.110352549...|       1.0|[3.66319522778508...|
|                          Yes|               0.05|        10.37|                 RENT|  0.0|[-19872.583933128...|       0.0|            [1.0,0.0]|
|                          Yes|               0.08|        13.92|                 RENT|  0.0|[-33638.923443237...|       0.0|            [1.0,0.0]|
|                          Yes|               0.12|        11.01|                 RENT|  0.0|[-16051.181108023...|       0.0|[1.0,3.2406310020...|
|                          Yes|               0.04|        10.59|                 RENT|  0.0|[-12695.327189993...|       0.0|            [1.0,0.0]|
|                          Yes|               0.19|        14.27|                 RENT|  0.0|[-32398.475124906...|       1.0|            [0.0,1.0]|
|                          Yes|               0.09|         9.99|                 RENT|  0.0|[-21347.784804152...|       0.0|            [1.0,0.0]|
|                          Yes|               0.15|        14.96|                 RENT|  0.0|[-29867.687590577...|       1.0|[2.07643973250091...|
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
only showing top 10 rows

Test Area Under ROC 0.532467744468951
```

8.3 Test: Random forest

```python
# Test: Random Forrest Model with test data
prediction_random_forrest = rfModel.transform(test)
prediction_random_forrest.select('previous_loan_defaults_on_file', 'loan_percent_income', 'loan_int_rate', 'person_home_ownership', 'label', 'rawP
from pyspark.ml.evaluation import BinaryClassificationEvaluator
valuator = BinaryClassificationEvaluator()
print('Test Area Under ROC', evaluator.evaluate(prediction_random_forrest))
```

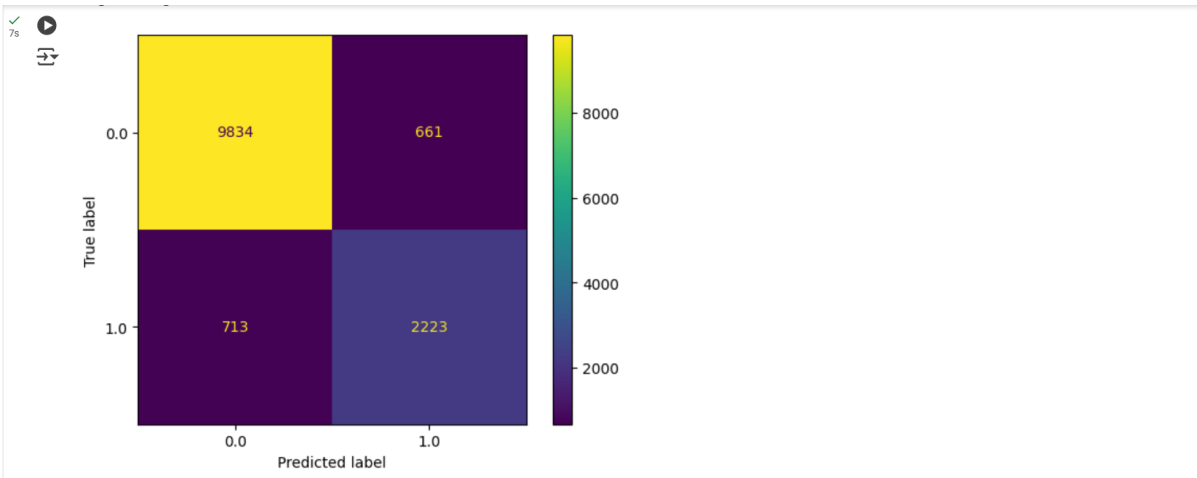
```
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
|previous_loan_defaults_on_file|loan_percent_income|loan_int_rate|person_home_ownership|label|       rawPrediction|prediction|         probability|
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
|                          Yes|               0.14|        15.02|                 RENT|  0.0|[17.2129743949608...|       0.0|[0.86064871974804...|
|                          Yes|               0.09|         6.99|                 RENT|  0.0|[18.4201469533737...|       0.0|[0.92100734766868...|
|                          Yes|               0.16|        10.56|                 RENT|  0.0|[18.2473415944277...|       0.0|[0.91236707972138...|
|                          Yes|               0.05|        10.37|                 RENT|  0.0|[18.4301150069153...|       0.0|[0.92150575034576...|
|                          Yes|               0.08|        13.92|                 RENT|  0.0|[18.4201469533737...|       0.0|[0.92100734766868...|
|                          Yes|               0.12|        11.01|                 RENT|  0.0|[18.1391043297783...|       0.0|[0.90695521648891...|
|                          Yes|               0.04|        10.59|                 RENT|  0.0|[18.4201469533737...|       0.0|[0.92100734766868...|
|                          Yes|               0.19|        14.27|                 RENT|  0.0|[17.2129743949608...|       0.0|[0.86064871974804...|
|                          Yes|               0.09|         9.99|                 RENT|  0.0|[18.4301150069153...|       0.0|[0.92150575034576...|
|                          Yes|               0.15|        14.96|                 RENT|  0.0|[17.2129743949608...|       0.0|[0.86064871974804...|
+-----------------------------+-------------------+-------------+---------------------+-----+--------------------+----------+--------------------+
only showing top 10 rows

Test Area Under ROC 0.958333652459391
```

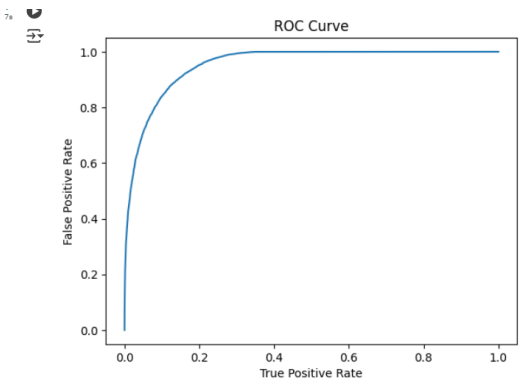
9. ประเมินประสิทธิภาพของ Model

9.1 Performance: Logistic Regression

```python
[41]
# Performance: Logistic Regression

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,classification_report
from pyspark.mllib.evaluation import MulticlassMetrics
trainingSummary = lrModel.summary

# Collect Predicted values and actual values
y_true = prediction_logistic_regression_model.select(['label']).collect()
y_pred = prediction_logistic_regression_model.select(['prediction']).collect()
labels = np.array(list(set(y_true)) )
print(classification_report(y_true, y_pred))

# Show Confusion Matrix
cm = confusion_matrix(y_true, y_pred, labels=labels[:,0])
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=labels[:,0])
disp.plot()
plt.show()

roc = trainingSummary.roc.toPandas()
plt.plot(roc['FPR'],roc['TPR'])
plt.ylabel('False Positive Rate')
plt.xlabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()
```

```
              precision    recall  f1-score   support

         0.0       0.93      0.94      0.93     10495
         1.0       0.77      0.76      0.76      2936

    accuracy                           0.90     13431
   macro avg       0.85      0.85      0.85     13431
weighted avg       0.90      0.90      0.90     13431
```



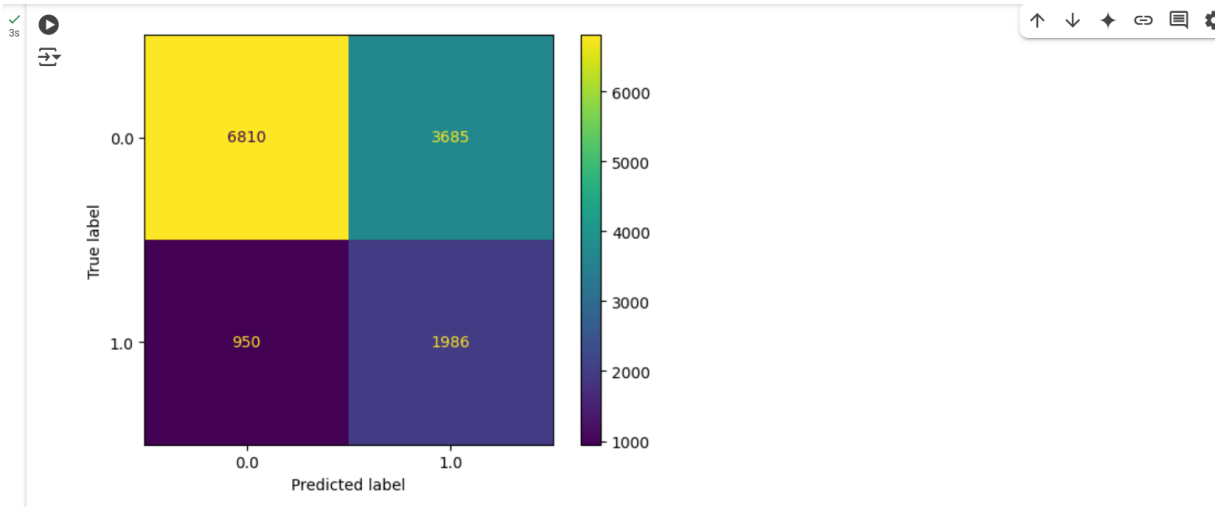


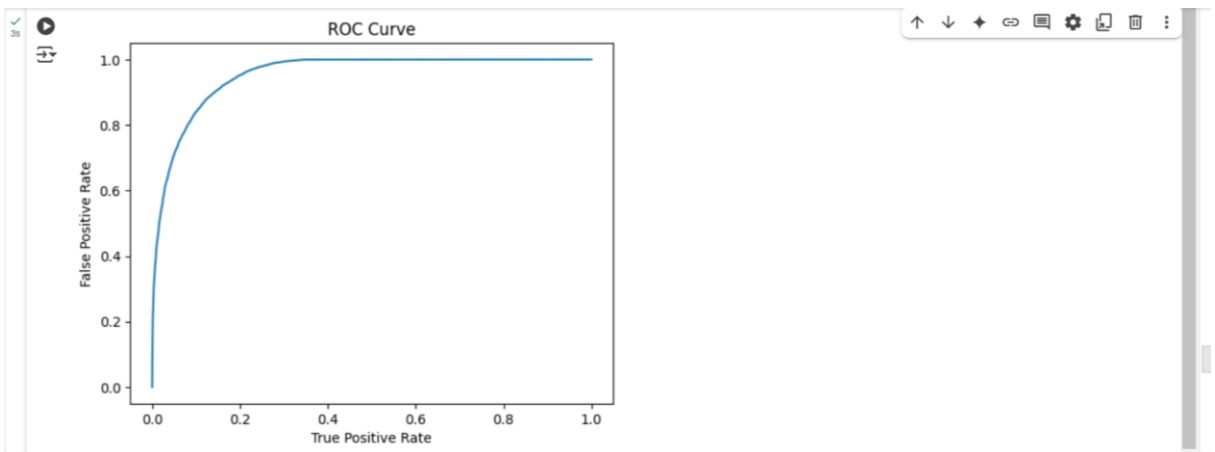
โมเดล Logistic Regression นี้มีความแม่นยำโดยรวม 90% ทำนายคลาส 0 "ไม่อนุมัติเงินกู้" ได้ดี แต่ยังมีข้อจำกัดในการทำนายคลาส 1 "อนุมัติเงินกู้" ซึ่ง Recall ต่ำ 76% อาจต้องปรับปรุงเพื่อให้เหมาะสมกับข้อมูลที่ Unbalance โดยการ Rebalance

9.2 Performance: Naive Bayes

```python
# Performance: Naive Bayes

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,classification_report
from pyspark.mllib.evaluation import MulticlassMetrics

# Collect Predicted values and actual values
y_true = prediction_native_bayes.select(['label']).collect()
y_pred = prediction_native_bayes.select(['prediction']).collect()
labels = np.array(list(set(y_true)) )
print(classification_report(y_true, y_pred))

# Show Confusion Matrix
cm = confusion_matrix(y_true, y_pred, labels=labels[:,0])
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=labels[:,0])
disp.plot()
plt.show()

roc = trainingSummary.roc.toPandas()
plt.plot(roc['FPR'],roc['TPR'])
plt.ylabel('False Positive Rate')
plt.xlabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()
```

```
              precision    recall  f1-score   support

         0.0       0.88      0.65      0.75     10495
         1.0       0.35      0.68      0.46      2936

    accuracy                           0.65     13431
   macro avg       0.61      0.66      0.60     13431
weighted avg       0.76      0.65      0.68     13431
```





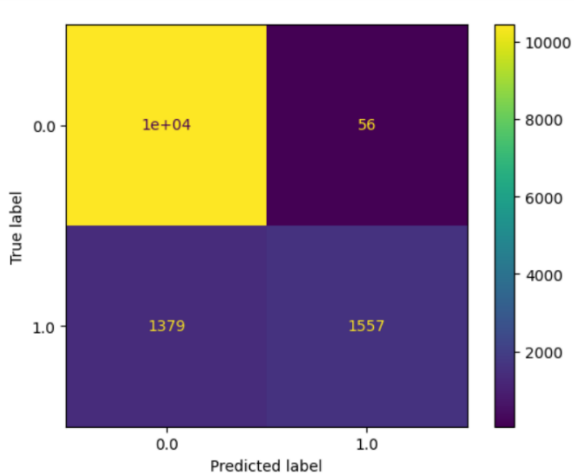
โมเดล Naive Bayes นี้มีความแม่นยำโดยรวม (Accuracy) ที่ 65% แต่ประสิทธิภาพในการทำนาย class `1` (การอนุมัติเงินกู้) ต่ำ เนื่องจากข้อมูลไม่สมดุล ทำให้ precision และ F1-score ของ class `1` ต่ำ โมเดลเหมาะกับ class `0` (ไม่อนุมัติ) มากกว่า สามารถปรับปรุงได้ด้วยการเน้นจัดการปัญหาความไม่สมดุลของข้อมูล.

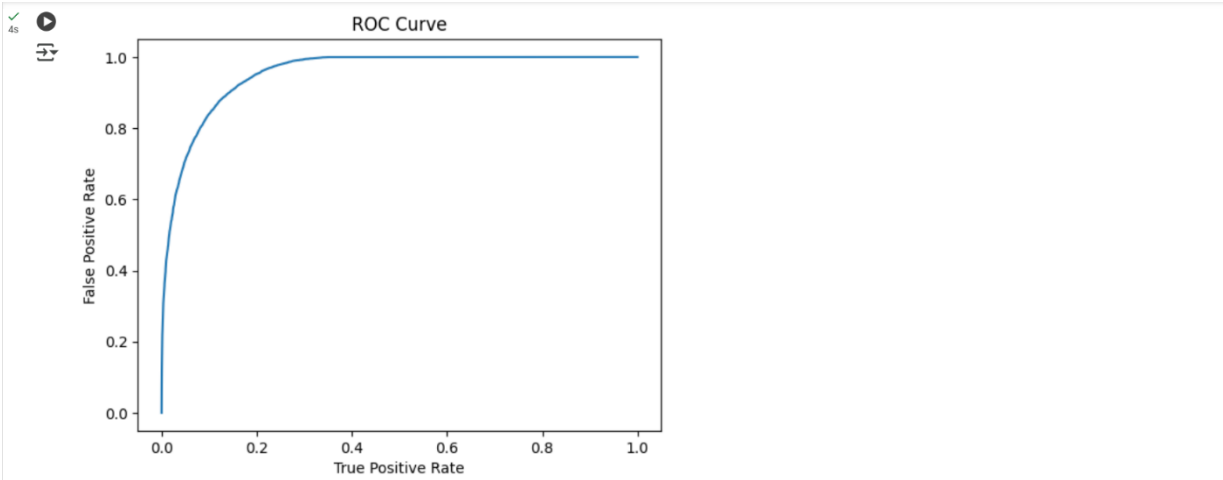
9.3 Performance: Random forest

[43]

```python
# Performance: Random Forest

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,classification_report
from pyspark.mllib.evaluation import MulticlassMetrics

# Collect Predicted values and actual values
y_true = prediction_random_forrest.select(['label']).collect()
y_pred = prediction_random_forrest.select(['prediction']).collect()
labels = np.array(list(set(y_true)) )
print(classification_report(y_true, y_pred))

# Show Confusion Matrix
cm = confusion_matrix(y_true, y_pred, labels=labels[:,0])
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=labels[:,0])
disp.plot()
plt.show()

roc = trainingSummary.roc.toPandas()
plt.plot(roc['FPR'],roc['TPR'])
plt.ylabel('False Positive Rate')
plt.xlabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()
```

```
              precision    recall  f1-score   support

         0.0       0.88      0.99      0.94     10495
         1.0       0.97      0.53      0.68      2936

    accuracy                           0.89     13431
   macro avg       0.92      0.76      0.81     13431
weighted avg       0.90      0.89      0.88     13431
```





โมเดล Random Forest นี้มีความแม่นยำโดยรวม 89% และทำนาย Class `0` (ไม่ผ่านการอนุมัติ) ได้ดี (Recall = 99%) แต่ยังมีปัญหาในการทำนาย Class `1` (ผ่านการอนุมัติ) ซึ่ง Recall ต่ำเพียง 53% เนื่องจากข้อมูลมีความไม่สมดุล สามารถปรับสมดุลข้อมูลหรือปรับพารามิเตอร์เพื่อเพิ่มประสิทธิภาพ.